Week: 1

DAY 1 – AGENT FOUNDATIONS

Reasoning Loops, Tool Calling & Raw Python Agents

1. Introduction

Artificial Intelligence agents are systems that can receive a user request, reason about the required actions, use external tools when necessary, observe the results, and continue working until the requested task is completed. Unlike a simple chatbot that only generates text, an agent can interact with tools and use the results of those tools to produce a more useful and reliable response.

The main purpose of Day 5 was to understand the fundamental architecture of an AI agent by implementing one from scratch using raw Python. Instead of relying on frameworks such as LangChain or LangGraph, the agent was developed manually so that the complete process of model interaction, tool calling, execution, memory, state management, logging, and error handling could be understood.

The implementation used the Google Gemini API with the Gemini 2.5 Flash model. Two simple tools were created for demonstration:

- A calculator tool for arithmetic operations.
- A weather tool using a simulated weather dataset.

The agent was designed to identify when a tool was required, generate a function call, execute the selected tool, provide the result back to the model, and continue the loop until a final answer was produced.

The five tasks completed during this day were:

1. Understanding Agent Concepts and Architecture
2. Implementing Tool Calling
3. Building a Raw Python Agent Loop
4. Implementing Memory, State and Logging
5. Testing Failure Modes and Adding Guardrails

This practical implementation helped demonstrate how modern AI agents work internally and why agent frameworks are useful for larger and more complex systems.

2. Scenario

The scenario for this task was to build a minimal AI agent from scratch using raw Python.

The agent needed to demonstrate the following capabilities:

- Understand a user request.
- Decide whether a tool is required.
- Select the appropriate tool.
- Generate arguments for the selected tool.
- Execute the tool.
- Observe the tool result.
- Continue reasoning based on the observation.
- Produce a final response.
- Maintain useful working state.
- Record important events through logging.
- Handle tool failures safely.
- Stop after a maximum number of iterations.

The implementation intentionally avoided high-level agent frameworks. This allowed the basic agent architecture and reasoning loop to be implemented explicitly.

3. Task 1 – Understanding Agent Concepts and Architecture

Objective

The objective of Task 1 was to understand the basic components of an AI agent and how those components work together.

What is an AI Agent?

An AI agent is a system that can perceive a task, decide what action is required, interact with tools or external systems, observe the results, and continue working toward a goal.

A basic agent can be represented as:

User Request → AI Model → Tool Selection → Tool Execution → Observation → AI Model → Final Answer

The model acts as the reasoning component, while tools provide capabilities that the model cannot reliably perform by itself.

For example, if a user asks:

"What is 25 multiplied by 8?"

The model can decide that a calculator tool should be used. The calculator executes the multiplication and returns 200. The model then uses that result to provide the final answer.

Main Components of an Agent

1. User Input

The process begins when the user provides a request or task.

Example:

What is 25 multiplied by 8?

2. AI Model

The language model interprets the request and determines what action should be taken.

3. Tools

Tools are external functions that provide additional capabilities to the agent.

Examples include:

- Calculator
- Weather API
- Database
- Search engine
- File system
- External applications

4. Tool Selection

The model determines whether a tool is required and selects the appropriate tool.

5. Tool Execution

The selected function is executed using the arguments generated by the model.

6. Observation

The tool returns a result. The result becomes an observation that can be provided back to the model.

7. Final Answer

After obtaining sufficient information, the model generates a final response for the user.

Agent Architecture

The implemented architecture followed this general process:

User Request

↓

Gemini Model

↓

Determine Required Action

↓

Function Call

↓

Tool Execution

↓

Tool Result / Observation

↓

Gemini Model

↓

Final Answer

This architecture demonstrates the basic reasoning-action-observation cycle used by many modern agent systems.

Task 1 Result

Task 1 successfully established the conceptual foundation required to build the agent. The difference between a normal chatbot and a tool-using agent was also demonstrated.

A normal chatbot mainly generates responses, whereas an agent can interact with external functions and use their results before generating the final response.

4. Task 2 – Tool Calling

Objective

The objective of Task 2 was to implement function/tool calling using the Gemini API.

Tool calling allows the language model to request execution of a predefined function instead of attempting to perform every operation itself.

Calculator Tool

A calculator function was created to perform four basic operations:

- Addition
- Subtraction
- Multiplication
- Division

The function accepts three arguments:

- a
- b
- operation

The function also contains validation for division by zero and unknown operations.

For example:

Input:

25, 8, multiply

Tool Result:

200

Weather Tool

A second tool was created to simulate weather information.

The weather dataset contained sample information for:

- Karachi
- Lahore
- Islamabad
- Dubai

Each city had:

- Temperature
- Weather condition

For example, Karachi was represented with a temperature of 34°C and a Sunny condition.

The weather function also validates the requested city. If information for a city is unavailable, the function raises an error.

Tool Registry

A tool registry was created to keep track of available functions.

The registry included:

- calculator
- get_weather

This approach allows the agent to check whether a requested tool actually exists before attempting to execute it.

Function Declaration

The calculator tool was described to Gemini using a function declaration. The declaration specified:

- Tool name
- Tool description
- Input parameters
- Parameter types
- Required parameters
- Allowed operations

This information helps the language model understand how and when the tool should be used.

Manual Tool Calling Process

The tool-calling process followed these steps:

1. A user request was created.
2. The request was sent to Gemini.
3. Gemini analyzed the request.
4. Gemini generated a function call.
5. The function name was extracted.
6. Function arguments were extracted.
7. The calculator function was executed.
8. The result was returned to Gemini.
9. Gemini generated the final response.

Example

User request:

"What is 25 multiplied by 8?"

Gemini identified that the calculator tool was required.

Function call:

calculator(a=25, b=8, operation="multiply")

Tool execution:

25 × 8 = 200

The result was then provided back to Gemini, which generated the final natural-language response.

Task 2 Result

Task 2 successfully demonstrated function calling. The model did not directly invent the arithmetic result. Instead, it selected the calculator tool, supplied the required arguments, received the tool result, and then generated the final answer.

This demonstrated how external tools extend the capabilities of language models.

5. Task 3 – Building a Raw Python Agent Loop

Objective

The objective of Task 3 was to convert the manual tool-calling process into a reusable agent loop using raw Python.

A complete agent should not stop after one tool call. It should be capable of repeatedly interacting with tools until the task is complete.

Agent Loop

The implemented agent followed a loop similar to:

Think → Act → Observe → Continue

The actual implementation was based on model responses and tool calls rather than a fixed sequence of actions.

The loop performed the following steps:

1. Send the current messages to the model.
2. Inspect the model response.
3. Check whether a function call was generated.
4. If no function call exists, return the final response.
5. If a function call exists, identify the tool.
6. Extract tool arguments.
7. Execute the corresponding Python function.
8. Store the result.
9. Add the tool result to the conversation.
10. Call the model again.
11. Repeat until a final answer is produced.

Maximum Iteration Limit

A maximum iteration limit was added to prevent infinite loops.

The agent was configured with a maximum number of iterations.

This is an important guardrail because an agent could potentially continue calling tools repeatedly if there is no stopping condition.

For example:

Iteration 1 → Tool Call

Iteration 2 → Tool Call

Iteration 3 → Tool Call

If the task is still not complete, the agent continues only until the configured maximum number of iterations is reached.

Multi-Step Reasoning

The raw agent loop was also tested with tasks requiring more than one action.

This demonstrated that the agent was not simply a single function-call script. It could:

- Receive a request.
- Perform an action.
- Observe the result.
- Use the result for the next action.
- Continue until completion.

This is one of the most important differences between basic function calling and an actual agent loop.

ReAct-Style Behaviour

The implementation demonstrated the basic idea behind ReAct-style agent behaviour:

Reason → Act → Observe → Reason → Act → Observe → Final Answer

The model determines the required action, a tool performs the action, the result is observed, and the model decides what to do next.

The implementation did not use a ReAct framework. Instead, the loop was manually implemented in Python to understand the underlying mechanism.

Task 3 Result

Task 3 successfully converted individual function calling into a reusable agent loop.

The implementation supported:

- Tool detection
- Tool selection

- Argument extraction
- Tool execution
- Tool results
- Repeated iterations
- Multi-step tasks
- Final response generation
- Maximum iteration protection

This demonstrated the core execution cycle of an AI agent.

6. Task 4 – Memory, State and Logging

Objective

The objective of Task 4 was to add memory, working state, and logging to the agent.

A useful agent needs more than a model and tools. It also needs to keep track of what it is currently doing, what actions have already been completed, and what results have been obtained.

Conversation Memory

Conversation memory was implemented using a Python list.

Each message was stored with:

- Role
- Content

For example:

user: What is 25 multiplied by 8?

assistant: The calculator result is 200.

This allows the system to maintain a history of interactions.

Working Memory

A separate working-memory structure was created to store the agent's current execution state.

It tracked:

- Current task
- Completed steps

- Tool results
- Current iteration

This information is different from normal conversation history.

Conversation memory stores what was said, while working memory stores what the agent is currently doing and what it has already accomplished.

Example Working Memory

The working memory could contain information such as:

Current Task: Find the weather information.

Steps Completed: Weather tool executed.

Tool Results: Temperature and weather condition.

Iteration: 2

This state helps the agent maintain a structured view of its current task.

Logging

An agent log was also implemented.

The log records important events during execution.

Examples include:

- START
- ITERATION
- TOOL_CALL
- TOOL_RESULT
- TOOL_ERROR
- FINISH
- STOP

For example:

START: User request received.

ITERATION: 1

TOOL_CALL: calculator

TOOL_RESULT: 200

FINISH: Final response generated.

Why Logging is Important

Logging is important for debugging and monitoring AI agents.

Without logs, it can be difficult to determine:

- Which tool was selected.
- Which arguments were passed.
- Which result was returned.
- Where an error occurred.
- How many iterations were performed.
- Why the agent stopped.

The logging system therefore provides visibility into the internal execution process.

Integration with Agent Loop

The most important part of Task 4 was integrating memory and logging directly into the actual agent loop.

The agent updates the working memory at every iteration.

When a tool is called, the event is logged.

When the tool returns a result, that result is stored.

If an error occurs, the error is also recorded.

When the agent completes the task, a final event is logged.

Therefore, memory and logging were not only created as separate demonstration functions; they were connected to the actual agent execution.

Task 4 Result

Task 4 successfully demonstrated how memory, working state, and logging can make an agent easier to understand, debug, and control.

The implementation tracked the current task, iterations, completed actions, tool results, and execution events.

7. Task 5 – Failure Modes and Guardrails

Objective

The objective of Task 5 was to deliberately create an agent failure and demonstrate how guardrails can prevent the entire program from crashing.

Reliable AI agents must be designed to handle unexpected situations because tools can fail, inputs can be invalid, and models can select unavailable tools.

Deliberate Failure Test

A failure test was performed by requesting weather information for Tokyo.

The weather dataset intentionally did not contain Tokyo.

The request was:

"Find the weather in Tokyo and tell me its temperature."

The weather tool checked the available dataset and determined that Tokyo was unavailable.

Instead of allowing the Python exception to crash the notebook, exception handling was added to the agent loop.

The tool failure was captured and recorded as a tool error.

Error Handling

The agent used try/except blocks around tool execution.

If a tool fails, the system:

1. Catches the exception.
2. Converts it into an error message.
3. Records the error in working memory.
4. Adds the error to the agent logs.
5. Displays the error safely.
6. Allows the agent to continue or finish without an uncontrolled traceback.

This is an important reliability feature.

Failure Modes

Several common agent failure modes were identified.

1. Infinite Loops

An agent could repeatedly call tools without reaching a final answer.

Mitigation:

A maximum iteration limit was implemented.

2. Wrong Tool Selection

The model may select an inappropriate tool.

Mitigation:

Clear tool names and descriptions should be provided.

3. Invalid Arguments

The model may provide incorrect or incomplete arguments.

Mitigation:

Structured function schemas, required parameters, and validation should be used.

4. Tool Errors

An external tool may fail or may not have the requested information.

Mitigation:

Exception handling should be implemented around tool execution.

5. Unknown Tools

The model may request a tool that does not exist in the registry.

Mitigation:

The agent validates the tool name against the available tool registry before execution.

6. Hallucinated Results

A language model may attempt to answer using generated information instead of using a required tool.

Mitigation:

The system should clearly instruct the model to use tools when reliable external information is required.

Guardrails Implemented

The following guardrails were implemented or demonstrated:

- Maximum iteration limit

- Tool registry validation
- Exception handling
- Tool error reporting
- Input validation
- Structured function declarations
- Logging
- Working-memory tracking

These guardrails improve reliability and make the agent safer to operate.

Why Agent Frameworks Exist

The raw Python implementation demonstrates that an agent requires several components:

- Model interaction
- Tool definitions
- Tool execution
- Message history
- Memory
- State management
- Error handling
- Logging
- Iteration control
- Stopping conditions

For small experiments, these components can be implemented manually.

However, as applications become larger, managing all of these components manually becomes difficult. Frameworks such as LangChain, LangGraph, and CrewAI provide reusable abstractions for tools, state, workflows, memory, routing, retries, and multi-agent systems.

Understanding the raw implementation first is valuable because it makes the behaviour of these frameworks easier to understand.

Task 5 Result

The deliberate Tokyo weather failure demonstrated that an agent cannot assume every tool request will succeed.

The tool generated an error because the requested city was not available. The error was caught by the agent and handled safely instead of causing an uncontrolled notebook crash.

This demonstrated why failure testing and guardrails are essential when building reliable AI agents.

---

8. Overall Implementation Summary

The five tasks together produced a complete minimal agent architecture.

The final workflow can be summarized as:

User Request

↓

Gemini 2.5 Flash

↓

Determine Whether a Tool is Required

↓

Select Tool

↓

Generate Arguments

↓

Execute Python Tool

↓

Store Result

↓

Log Event

↓

Return Observation to Model

↓

Continue Agent Loop

↓

Generate Final Answer

The implementation also maintained:

- Conversation memory
- Working memory
- Tool results
- Iteration count
- Execution logs
- Error information
- Maximum iteration protection

This created a small but functional agent system without depending on a high-level agent framework.

9. Key Learning Outcomes

After completing Day 5, the following concepts were demonstrated:

Agent Architecture

An agent combines an AI model, tools, memory, state, and control logic.

Tool Calling

Language models can select predefined functions and provide structured arguments for those functions.

Agent Loops

Agents can repeatedly perform actions, observe results, and continue until a task is completed.

Memory

Conversation memory preserves interaction history, while working memory tracks the current execution state.

Logging

Logs provide visibility into the internal behaviour of an agent and help with debugging.

Error Handling

Tools can fail, so agents need controlled exception handling instead of assuming every operation succeeds.

Guardrails

Iteration limits, validation, tool registries, schemas, and error handling improve reliability.

Frameworks

Agent frameworks provide abstractions for the same fundamental components implemented manually in this task.

10. Final Conclusion

Day 5 provided a practical understanding of how AI agents operate internally. Instead of treating an AI agent as a black box, the complete process was implemented using raw Python and the Gemini API.

The first task established the fundamental concepts and architecture of agents. The second task demonstrated function calling through calculator and weather tools. The third task converted individual tool calls into a reusable agent loop capable of performing multiple actions. The fourth task introduced conversation memory, working memory, state tracking, and execution logging. Finally, the fifth task deliberately tested a failure case and demonstrated the importance of error handling and guardrails.

The final implementation successfully demonstrated the core agent cycle:

Reason → Act → Observe → Continue → Final Answer

The project also showed that reliable agent systems require more than a powerful language model. Tools, structured inputs, memory, state management, logging, validation, exception handling, and stopping conditions are all important components of a robust agent.

Building the agent without using frameworks provided a deeper understanding of what happens behind the scenes in modern agent frameworks. Frameworks such as LangChain and LangGraph can simplify these components in larger applications, but understanding the raw Python implementation provides a strong foundation for designing and debugging more advanced AI systems.

Overall, Day 1 successfully achieved its objective of demonstrating **Agent Foundations, Reasoning Loops, Tool Calling, Memory, State Management, Logging, Failure Handling, and Guardrails** through a practical raw-Python implementation.